Lab2

2026年2月23日 星期一 21:20

1. SmartCard Class

Code:

```java
public class SmartCard {  7 个用法
    private String ownerName; // stores the full name of the card owner  2 个用法
    private boolean isStaff; // true if the cardholder is staff, false if student  3 个用法

    public SmartCard(String ownerName) {  2 个用法
        this.ownerName = ownerName; // 'this.ownerName' refers to the field, 'ownerName' (right) is the parameter
        this.isStaff = false; // Every new card defaults to student status
    }

    public String getOwner() {  0 个用法
        return ownerName; // Returns the stored owner name string to the caller
    }

    public boolean isStaff() {  0 个用法
        return isStaff; // Returns the current boolean value of the isStaff field
    }

    public void setStaff(boolean isStaff) {  1 个用法
        this.isStaff = isStaff; // 'this.isStaff' is the field, 'isStaff' (right) is the method parameter, 'this' distinguishes them
    }
}
```

2. CardLock Class

Code:

```java
public class CardLock { // Public class to be used by Door, Simulation  5 个用法

    private SmartCard lastCardSeen; // Private field to hold a reference to the most recently swiped SmartCard object  4 个用法
    private boolean allowStudents;  // students can unlock the door when true  4 个用法

    public CardLock() {  2 个用法
        this.allowStudents = false; // Only staff may unlock by default; students are blocked until toggleStudentAccess() is called
    }

    // Stores the reference to the passed SmartCard so isUnlocked() can check it later
    public void swipeCard(SmartCard card) {  2 个用法
        this.lastCardSeen = card; // Overwrites any previously swiped card; only the LAST card matters for unlock checks
    }

    public SmartCard getLastCardSeen() {  0 个用法
        return lastCardSeen; // Returns the reference to the last SmartCard
    }

    public void toggleStudentAccess() {  0 个用法
        allowStudents = !allowStudents; // inverts the current value
    }

    public boolean isUnlocked() {  1 个用法
        // First check: impossible to get a NullPointerException if no card was swiped yet
        // Second check (inside parentheses): uses || (OR) so EITHER condition being true is enough to unlock
        return lastCardSeen != null && (lastCardSeen.isStaff() || allowStudents);
        // guard against null / staff card grants access / student access mode also grants access
    }
}
```

3. Simulation class

Code:

```java
public class Simulation {
    public static void main(String[] args) {
        Door researchLab = new Door( roomName: "Research Labs");  // Create a Door object, the Door constructor also creates a default CardLock

        CardLock labLock = new CardLock(); // Constructs a new CardLock with allowStudents = false by default

        researchLab.attachLock(labLock); // Passes the labLock reference to the Door; Door and Simulation now share the SAME CardLock object

        SmartCard studentCard = new SmartCard( ownerName: "Student A"); // Constructor sets isStaff = false
        SmartCard staffCard   = new SmartCard( ownerName: "Staff B");   // Also starts as false; we must explicitly promote it to staff below

        staffCard.setStaff(true); // Sets isStaff field inside staffCard to true; this card can now unlock staff-only doors

        labLock.swipeCard(studentCard); // Registers studentCard as the lastCardSeen on the CardLock
        System.out.println("Door Open (Student)? " + researchLab.openDoor());
        // openDoor() to lock.isUnlocked() to studentCard.isStaff() returns false AND allowStudents is false so that it will return false

        labLock.swipeCard(staffCard); // Overwrites lastCardSeen with staffCard
        System.out.println("Door Open (Staff)? " + researchLab.openDoor());
        // openDoor() to lock.isUnlocked() to staffCard.isStaff() returns true so that overall returns true
    }
}
```

Output:

```
Door Open (Student)? false
Door Open (Staff)? true
```

Questions for Logbook

• What is the difference between a class and an object?

A class is like a blueprint or template, it defines what attributes and methods something should have, but it doesn't actually exist in memory yet. And object is a real instance created from that blueprint. For example, SmartCard is the class here, but when I write new SmartCard("Student A"), that creates an actual object in memory. I can also create many objects from the same class. In this lab, SmartCard is the class, while studentCard and staffCard are two separate objects made from it.

• Why use a constructor instead of direct variable assignment?

Because a constructor makes sure the object is set up correctly when it's created. If I just did direct assignment like:
javaSmartCard card = new SmartCard();
card.ownerName = "Student A";
I may forget to set one of the fields, making the object incomplete. With a constructor, all the required values are passed in at creation time so I can't accidentally skip them. In this lab, the SmartCard constructor automatically sets isStaff = false for every new card, so I don't have to remember to do that manually every time.

• What issues arise if isStaff is modified directly instead of using setStaff()?

If isStaff were public and could be changed directly like studentCard.isStaff = true, it might be set to an invalid state. Second, if I later wanted to add validation , I have to find and update every place in the code that does direct assignment, which could be a lot of places. By using setStaff(), all the logic is in one place. This is called encapsulation, keeping the fields private and control access through methods so the class can protect its own data.

2. Using ArrayLists

1. User Class

Code:

```java
public class User {  11 个用法

    private String username; // unique login identifier for this user  2 个用法
    private String userType; // "user", "editor", "admin"  3 个用法
    private String name; // human-readable display name of this user  2 个用法

    public User(String username, String userType, String name) {  4 个用法
        this.username = username; // Assigns the constructor parameter 'username' to the instance field 'this.username'
        this.userType = userType; // Assigns 'userType' parameter to the instance field; 'this.' distinguishes field from parameter
        this.name = name; // Assigns 'name' parameter to the instance field 'this.name'
    }

    public String getUsername() {  3 个用法

        return username; // Returns the stored username
    }

    public String getUserType() {  3 个用法

        return userType; // Returns the stored userType
    }

    public String getName() { return name; // Returns the stored name string }

    public void setUserType(String userType) {  0 个用法
        this.userType = userType; // Overwrites the current role; 'this.userType' = field, 'userType' = parameter
    }
}
```

2. UserGroup Class

Code:

```java
import java.util.ArrayList; // using ArrayList<>

public class UserGroup {  2 个用法

    private ArrayList<User> users; // Private dynamic list (ArrayList) that stores User objects  9 个用法

    public UserGroup() {  1 个用法
        users = new ArrayList<>(); // creating an empty list
    }

    public void addSampleData() {  1 个用法
        users.add(new User( username: "fj3",  userType: "user",   name: "Francis")); // Creates a User object inline and adds it to the list, index 0
        users.add(new User( username: "amn",  userType: "admin",  name: "Amna")); // Admin user, index 1
        users.add(new User( username: "cm2",  userType: "admin",  name: "Cameron")); // Admin user, index 2
        users.add(new User( username: "pm5",  userType: "editor", name: "Peter")); // Editor user, index 3
```

```java
    }

    public User getUser(int index) { return users.get(index); // retrieving the element at the given index 0 }

    public void printUsernames() {  1 个用法
        for (User u : users) {    // 'u' is a temporary variable referencing each User in turn
            System.out.println(u.getUsername() + " " + u.getUserType()); // Concatenates username and userType with a space
        }
    }

    public void removeUser(String username) {  0 个用法
        users.removeIf( User user -> user.getUsername().equals(username));
        // 'removeIf' goes through each element and removes it if the lambda (condition) returns true
        // 'user ->' declares a lambda parameter named 'user' (each element in the list)
        // '.equals()' performs a content comparison; '==' would compare object references, not string values
    }

    public ArrayList<User> getUsers() { return users; // Returns the reference to the internal ArrayList }
}
```

3. Main Java

Code:

```java
import java.util.ArrayList; // Import ArrayList to declare the 'administrators' list

public class Main {
    public static void main(String[] args) { // initialises an empty ArrayList UserGroup
        UserGroup group = new UserGroup(); // 'group' is a reference variable pointing to the new UserGroup object on the heap

        group.addSampleData(); // Calls addSampleData() on the 'group' object to populate and adds 4 User objects to internal list

        System.out.println("All Users:");

        group.printUsernames(); // Call printUsernames() to loop through every user and print "username userType"

        ArrayList<User> administrators = new ArrayList<>(); // Declares and initialises an empty list; <User> is the generic type parameter (type-safe)

        // Loop through every User in the group's list and check if they are an admin
        for (User user : group.getUsers()) {
            if (user.getUserType().equals("admin")) {  //compares the string content of userType
                administrators.add(user); // Adds the current admin User reference to admin list
            }
        }

        System.out.println("\nAdministrators:");

        // Loop through the filtered administrators list and print each admin's username and type
        for (User admin : administrators) {
            System.out.println(admin.getUsername() + " " + admin.getUserType()); // Prints "amn admin", then "cm2 admin"
        }
    }
}
```

Output:

```
All Users:
fj3 user
amn admin
cm2 admin
pm5 editor


Administrators:
amn admin
cm2 admin
```

3. HackerRank

1. Java Arraylist

Code:

```java
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {

    public static void main(String[] args) {
        /* Enter your code here. Read input from STDIN. Print output to STDOUT. Your class should be named Solution. */
```

```java
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        ArrayList<ArrayList<Integer>> lines = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            int count = sc.nextInt();
            ArrayList<Integer> row = new ArrayList<>();
            for (int j = 0; j < count; j++) {
                row.add(sc.nextInt());
            }
            lines.add(row);
        }

        int q = sc.nextInt();
        for (int i = q; i > 0; i--) {
            int x = sc.nextInt() - 1;
            int y = sc.nextInt() - 1;

            if (x < 0) {
                System.out.println("ERROR!");
            } else if (x >= n) {
                System.out.println("ERROR!");
            } else if (y < 0) {
                System.out.println("ERROR!");
            } else if (y >= lines.get(x).size()) {
                System.out.println("ERROR!");
            } else {
                System.out.println(lines.get(x).get(y));
            }

        }
        sc.close();
    }
}
```

Output:

Submitted a few seconds ago • Score: 1.00 Status: Accepted

✔ Test Case #0 ✔ Test Case #1 ✔ Test Case #2
✔ Test Case #3 ✔ Test Case #4 ✔ Test Case #5

Testcase 0 ✔ Testcase 1 ✔

Congratulations, you passed the sample test case.

Click the **Submit Code** button to run your code against all the test cases.

Input (stdin)

```
5
5 41 77 74 22 44
1 12
4 37 34 36 52
0
3 20 22 33
5
1 3
3 4
3 1
4 3
5 5
```

Your Output (stdout)

```
74
52
37
ERROR!
ERROR!
```

Expected Output

```
74
52
37
ERROR!
ERROR!
```

2. Java Iterator

Code:

```
7
8           Object element = it.next(); // get the next element
9           if(element instanceof String && element.equals("###")) // stop exactly when I hit "###"
```

Output:

Congratulations, you passed the sample test case.

Click the **Submit Code** button to run your code against all the test cases.

Input (stdin)

```
2 2
42
10
hello
java
```

Your Output (stdout)

```
hello
java
```

Expected Output

```
hello
java
```

Submitted a minute ago • Score: 1.00 Status: Accepted

Test Case #0 Test Case #1 Test Case #2